PYTHON FOR ACTUARIES · NOTEBOOK 06

Machine Learning for Actuaries

A 90-Minute Practical Workshop

In this session, you will learn how a model learns from training data and predicts new data. You will also meet the four models of the fraud detection exercise in Notebook 07.

- **Part 1**: Training and Predicting
- **Part 2**: Four Models, Same Steps
- **Part 3**: Training Data vs. Test Data

The explanations use the small tables `claims` and `new_claims`. All exercises use `vehicle_fraud.csv` from Notebook 03.

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

# Avoid truncation of DataFrame output in Jupyter
pd.set_option('display.expand_frame_repr', False)

# Display all columns in the DataFrame output
pd.set_option('display.max_columns', None)

# Define a separator for better readability
sep = "=" * 50
```

Part 1: Training and Predicting

A model learns from past claims whose outcome is known: the **training data**. It then predicts the outcome of new claims: the **test data**. In the exercises, `train_test_split()` creates both sets (Notebook 05).

1.1 Past and New Claims

`claims` holds 16 past claims, `new_claims` four new ones. The features are already prepared as in Notebook 05. The outcome `fraud` of the new claims is known here, so the predictions can be checked.

```
# Past claims: the training data
claims = pd.DataFrame({
    'claim_value_ratio': [0.08, 0.12, 0.45, 0.10, 0.25, 0.06, 0.38, 0.15, 0.09, 0.52, 0.18,
        0.11, 0.07, 0.30, 0.14, 0.20],
    'reporting_delay_days': [5, 12, 38, 8, 15, 3, 30, 20, 6, 42, 10, 25, 4, 18, 9, 33],
    'police_report': [1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1],
    'fraud': [0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0]
})

print(f"\n{sep}\nPast Claims: claims\n{sep}")
print(claims)
```

```
==================================================
Past Claims: claims
==================================================
    claim_value_ratio  reporting_delay_days  police_report  fraud
0                0.08                     5              1      0
1                0.12                    12              1      0
2                0.45                    38              0      1
3                0.10                     8              1      0
4                0.25                    15              1      0
5                0.06                     3              1      0
6                0.38                    30              0      1
7                0.15                    20              0      0
8                0.09                     6              1      0
9                0.52                    42              0      1
10               0.18                    10              1      0
11               0.11                    25              1      1
12               0.07                     4              1      0
13               0.30                    18              0      1
14               0.14                     9              1      0
15               0.20                    33              1      0
```

```
# New claims: the test data
new_claims = pd.DataFrame({
    'claim_value_ratio': [0.48, 0.10, 0.22, 0.16],
    'reporting_delay_days': [35, 7, 28, 14],
    'police_report': [0, 1, 1, 1],
    'fraud': [1, 0, 0, 1]
})

print(f"\n{sep}\nNew Claims: new_claims\n{sep}")
print(new_claims)
```

```
==================================================
New Claims: new_claims
==================================================
   claim_value_ratio  reporting_delay_days  police_report  fraud
0               0.48                    35              0      1
1               0.10                     7              1      0
2               0.22                    28              1      0
3               0.16                    14              1      1
```

1.2 Features, Target and Scaling

As in Notebook 05, `X` holds the features and `y` the target `fraud`. `X_new` and `y_new` hold the same for the new claims. The scaler learns from the past claims and scales both tables.

```
# Features and target of the past claims
X = claims.drop(columns=['fraud'])
y = claims['fraud']

print(f"\n{sep}\nFeatures: X.columns.tolist()\n{sep}")
print(X.columns.tolist())
```

```
================================================
Features: X.columns.tolist()
================================================
['claim_value_ratio', 'reporting_delay_days', 'police_report']
```

```
# Features and target of the new claims
X_new = new_claims.drop(columns=['fraud'])
y_new = new_claims['fraud']
```

```
# Learn from the past claims, apply to both tables
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_new_scaled = scaler.transform(X_new)
```

1.3 Creating a Model

Logistic regression is a GLM for 0/1 targets: it turns a weighted sum of the features into a probability.

Creating a model sets its **hyperparameters** in parentheses: settings chosen before training. `max_iter=1000` allows enough steps for the computation, `random_state=42` makes the result reproducible. The new model has not learned anything yet.

```
# An untrained model with its hyperparameters
logreg = LogisticRegression(max_iter=1000, random_state=42)

print(f"\n{sep}\nModel: logreg\n{sep}")
print(logreg)
```

```
================================================
Model: logreg
================================================
LogisticRegression(max_iter=1000, random_state=42)
```

1.4 `fit()`: Learning from the Training Data

`fit()` learns from the features and the target of the training data. As with the `StandardScaler` in Notebook 05, the learned values are stored in the model, in attributes ending with `_`. Logistic regression learns one coefficient per feature: `coef_[0]`.

```
# Train the model on the past claims
logreg.fit(X_scaled, y)
print('Logistic Regression trained.')
```

```
Logistic Regression trained.
```

```python
# Learned coefficients with the feature names
coefficients = pd.Series(logreg.coef_[0], index=X.columns)

print(f"\n{sep}\nCoefficients: logreg.coef_[0]\n{sep}")
print(coefficients.round(2))
```

```
================================================
Coefficients: logreg.coef_[0]
================================================
claim_value_ratio       0.68
reporting_delay_days    0.82
police_report          -0.62
dtype: float64
```

A high claim-to-value ratio and a late report raise the fraud probability, a police report lowers it.

1.5 `predict()`: Predicting New Claims

`predict()` applies the learned coefficients to new claims and returns `1` (fraud) or `0` (no fraud) per claim. It needs only the features. New claims need the same preparation as the training data: here, the scaled features `X_new_scaled`.

```python
# Predict the new claims
y_pred = logreg.predict(X_new_scaled)

print(f"\n{sep}\nPredictions: logreg.predict(X_new_scaled)\n{sep}")
print(y_pred)
```

```
================================================
Predictions: logreg.predict(X_new_scaled)
================================================
[1 0 0 0]
```

1.6 `predict_proba()`: Predicting Probabilities

`predict_proba()` returns two columns per claim: the probability of `0` and the probability of `1`. Each row sums to 1.

```python
# Probabilities of 0 and 1 per claim
print(f"\n{sep}\nProbabilities: logreg.predict_proba(X_new_scaled)\n{sep}")
print(logreg.predict_proba(X_new_scaled).round(3))
```

```
================================================
Probabilities: logreg.predict_proba(X_new_scaled)
================================================
[[0.087 0.913]
 [0.938 0.062]
 [0.676 0.324]
 [0.877 0.123]]
```

`[:, 1]` selects the second column: the fraud probability. `predict()` turns it into `0` or `1` with the threshold 0.5.

```python
# Fraud probability per claim
y_proba = logreg.predict_proba(X_new_scaled)[:, 1]
```

```
# Fraud probability, prediction and actual outcome
comparison = pd.DataFrame({'fraud_probability': y_proba, 'prediction': y_pred, 'fraud':
    y_new})

print(f"\n{sep}\nComparison: fraud_probability, prediction, fraud\n{sep}")
print(comparison.round(3))
```

```
==================================================
Comparison: fraud_probability, prediction, fraud
==================================================
   fraud_probability  prediction  fraud
0              0.913           1      1
1              0.062           0      0
2              0.324           0      0
3              0.123           0      1
```

Claim 0 is flagged correctly. Claim 3 is fraud, but the model gives it only 12%: it looks like a legitimate claim.

Exercise 1: Training and Predicting on `vehicle_fraud.csv`

The first four code cells prepare the data in a shortened but clean way: they remove the duplicates, use only six numeric features without missing values and scale after the train/test split. Notebook 07 shows the full preparation from Notebook 05.

1. Create a logistic regression `lr_model` with `max_iter=1000` and `random_state=42`. Train it on the scaled training data.
2. Show the learned coefficients with the feature names, sorted. Which feature raises the fraud probability most? Which features lower it?
3. Predict the test data. How many claims does the model flag as fraud? How many test claims are fraudulent?
4. Compute the fraud probabilities of the test data. Show the five claims with the highest probability together with `fraud`.
5. A new claim: claim amount 1.5 times the vehicle value, reported after 35 days, customer for 1 year, 3 prior claims, no police report, no witnesses. Predict its fraud probability.

```
# Load the dataset and remove the duplicates
df = pd.read_csv('vehicle_fraud.csv')
df = df.drop_duplicates()
```

```
# Six features without missing values
df['claim_value_ratio'] = df['claim_amount'] / df['vehicle_value']
features = ['claim_value_ratio', 'reporting_delay_days', 'customer_since_years',
            'number_of_prior_claims', 'police_report', 'witnesses_present']
```

```
# 80% training data, 20% test data
X_train, X_test, y_train, y_test = train_test_split(df[features], df['fraud'],
    test_size=0.2, random_state=42, stratify=df['fraud'])
```

```
# Learn from the training data, apply to both sets
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Task 1: Logistic regression
# Hint: lr_model = LogisticRegression(max_iter=1000, random_state=42)
#       lr_model.fit(X_train_scaled, y_train)
```

```
# Task 2: Learned coefficients
# Hint: coefficients = pd.Series(lr_model.coef_[0], index=X_train.columns)
#       coefficients.sort_values()
```

```
# Task 3: Predictions
# Hint: y_pred = lr_model.predict(X_test_scaled)
#       y_pred.sum(), y_test.sum()
```

```
# Task 4: Fraud probabilities
# Hint: y_proba = lr_model.predict_proba(X_test_scaled)[:, 1]
#       comparison = pd.DataFrame({'fraud_probability': y_proba, 'fraud': y_test})
#       comparison.sort_values('fraud_probability', ascending=False).head()
```

```
# Task 5: Fraud probability of a new claim
# Hint: new_claim = pd.DataFrame({'claim_value_ratio': [1.5], 'reporting_delay_days': [35],
#                                 'customer_since_years': [1], 'number_of_prior_claims':
    [3],
#                                 'police_report': [0], 'witnesses_present': [0]})
#       new_claim_scaled = scaler.transform(new_claim)
#       lr_model.predict_proba(new_claim_scaled)[:, 1]
```

Part 2: Four Models, Same Steps

Every model is used the same way: create it, train it with `fit()`, predict with `predict()` and `predict_proba()`. Only the line that creates the model changes. Notebook 07 uses these four models:

Model	Idea	Features
`LogisticRegression`	GLM for 0/1 targets	scaled
`RandomForestClassifier`	Many decision trees that vote	unscaled
`XGBClassifier`	Decision trees that correct each other	unscaled
`MLPClassifier`	Neural network	scaled

As in Notebook 05: logistic regression and neural networks need scaled features, tree-based models do not.

2.1 Random Forest

A decision tree sorts the claims with yes/no questions, e.g. "reported after more than 20 days?". A random forest trains many trees, each on a random sample of the claims, and averages their votes. `n_estimators=100` sets the number of trees.

```
# Random forest with 100 trees
forest = RandomForestClassifier(n_estimators=100, random_state=42)
forest.fit(X, y)
print('Random Forest trained.')
```

```
Random Forest trained.
```

2.2 XGBoost

XGBoost also uses decision trees but builds them one after another: each new tree corrects the errors of the trees before it (**gradient boosting**). `n_estimators=100` sets the number of trees.

```
# XGBoost with 100 trees
boosting = XGBClassifier(n_estimators=100, random_state=42)
boosting.fit(X, y)
print('XGBoost trained.')
```

```
XGBoost trained.
```

2.3 Neural Network

A neural network passes the features through layers of neurons. `hidden_layer_sizes=(64, 32)` sets two layers with 64 and 32 neurons, `max_iter=500` limits the number of training rounds. `early_stopping=True` holds back 10% of the training data and stops training when the predictions there no longer improve.

```
# Neural network with two layers
network = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500, early_stopping=True,
    random_state=42)
network.fit(X_scaled, y)
print('Neural Network trained.')
```

```
Neural Network trained.
```

2.4 Comparing the Models

The dictionary `models` stores each model with the features it needs. The loop `for name, (model, X_eval) in models.items()` takes one model with its features at a time.

```
# Models with the features of the new claims
models = {
    'Logistic Regression': (logreg, X_new_scaled),
    'Random Forest': (forest, X_new),
    'XGBoost': (boosting, X_new),
    'Neural Network': (network, X_new_scaled),
}
```

```python
# Fraud probabilities of the new claims from every model
probas = {}
for name, (model, X_eval) in models.items():
    probas[name] = model.predict_proba(X_eval)[:, 1]
probas['fraud'] = y_new
probas_df = pd.DataFrame(probas)

print(f"\n{sep}\nFraud Probabilities of the New Claims\n{sep}")
print(probas_df.round(3))
```

```
================================================
Fraud Probabilities of the New Claims
================================================
   Logistic Regression  Random Forest  XGBoost  Neural Network  fraud
0                0.913           0.94    0.803           0.506      1
1                0.062           0.03    0.067           0.278      0
2                0.324           0.43    0.493           0.412      0
3                0.123           0.00    0.067           0.334      1
```

All four models flag claim 0 and miss claim 3, but they differ in how certain they are. The neural network is the least certain: 16 claims are far too few for it.

Exercise 2: Four Models on `vehicle_fraud.csv`

Continue with the data from Exercise 1. Use `random_state=42` for every model.

1. Train a random forest `rf_model` with `n_estimators=100`.
2. Train XGBoost `xgb_model` with `n_estimators=100`.
3. Train a neural network `mlp_model` with `hidden_layer_sizes=(64, 32)`, `max_iter=500` and `early_stopping=True`.
4. Store the four models with their test data in the dictionary `models`. Show the fraud probabilities of all four models for the first ten test claims together with `fraud`. Which fraud case do all four models miss?

```python
# Task 1: Random forest
# Hint: rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
#       rf_model.fit(X_train, y_train)
```

```python
# Task 2: XGBoost
# Hint: xgb_model = XGBClassifier(n_estimators=100, random_state=42)
#       xgb_model.fit(X_train, y_train)
```

```python
# Task 3: Neural network
# Hint: mlp_model = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500,
    early_stopping=True, random_state=42)
#       mlp_model.fit(X_train_scaled, y_train)
```

```
# Task 4: Fraud probabilities of all four models
# Hint: models = {
#           'Logistic Regression': (lr_model, X_test_scaled),
#           'Random Forest': (rf_model, X_test),
#           'XGBoost': (xgb_model, X_test),
#           'Neural Network': (mlp_model, X_test_scaled),
#       }
#       probas = {}
#       for name, (model, X_eval) in models.items():
#           probas[name] = model.predict_proba(X_eval)[:, 1]
#       probas['fraud'] = y_test
#       pd.DataFrame(probas).head(10)
```

Part 3: Training Data vs. Test Data

3.1 Accuracy

`accuracy_score(y_true, y_pred)` compares the predictions with the actual outcomes and returns the share of correct predictions.

```
# Share of correct predictions for the new claims
print(f"\n{sep}\nAccuracy: accuracy_score(y_new, logreg.predict(X_new_scaled))\n{sep}")
print(accuracy_score(y_new, logreg.predict(X_new_scaled)))
```

```
==================================================
Accuracy: accuracy_score(y_new, logreg.predict(X_new_scaled))
==================================================
0.75
```

The logistic regression predicts three of the four new claims correctly.

3.2 Learning by Heart

A model can learn its training data by heart (**overfitting**). Its accuracy on the training data then says little about new claims. The random forest predicts the past claims it was trained on, then the new claims.

```
# Accuracy on the past claims
print(f"\n{sep}\nAccuracy: accuracy_score(y, forest.predict(X))\n{sep}")
print(accuracy_score(y, forest.predict(X)))
```

```
==================================================
Accuracy: accuracy_score(y, forest.predict(X))
==================================================
1.0
```

```
# Accuracy on the new claims
print(f"\n{sep}\nAccuracy: accuracy_score(y_new, forest.predict(X_new))\n{sep}")
print(accuracy_score(y_new, forest.predict(X_new)))
```

```
==================================================
Accuracy: accuracy_score(y_new, forest.predict(X_new))
==================================================
0.75
```

On the past claims, the forest is perfect: it even recognizes claim 11, a fraud case that looks legitimate. On the new claims, it reaches 75%. Only data the model has not seen during training show how well it predicts. That is why every model is evaluated on the test data.

Exercise 3: Training Data vs. Test Data on `vehicle_fraud.csv`

Continue with `models` from Exercise 2.

1. Compute the accuracy of every model on the test data.
2. Compute the accuracy of the random forest on the training data. Compare it with Task 1.
3. A model that always predicts "no fraud" is right for every legitimate claim. What accuracy does it reach on the test data? Compare it with Task 1.

```
# Task 1: Accuracy on the test data
# Hint: for name, (model, X_eval) in models.items():
#           print(name, accuracy_score(y_test, model.predict(X_eval)))
```

```
# Task 2: Accuracy of the random forest on the training data
# Hint: accuracy_score(y_train, rf_model.predict(X_train))
```

```
# Task 3: Accuracy of a model that always predicts "no fraud"
# Hint: (y_test == 0).mean()
```

Summary

- **Training and predicting:** create a model with its hyperparameters, `fit()` on the training data, `predict()` and `predict_proba()` on new data with the same preparation
- **Four models:** `LogisticRegression`, `RandomForestClassifier`, `XGBClassifier` and `MLPClassifier`, all used the same way
- **Training vs. test data:** `accuracy_score()`, evaluation on the test data only

Next: detecting fraudulent claims in Notebook 07.

- scikit-learn: Getting Started
- XGBoost Documentation